BRACT's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Division: E Batch: 1 Practical No:10

Roll no: 38 PRN No:12413458 Name of Student: Vedanti Raut

Subject Name: Deep Learning

Problem Statement Design and implement a deep learning application for audio classification that incorporates Speech-to-Text (STT) and Text-to-Speech (TTS) using an appropriate deep learning architecture, and evaluate its performance using suitable metrics.

.

Algorithm:

1. Start the program and import the required libraries, including TensorFlow, PyTorch, Transformers, and Scikit-learn.
2. Download and load the Mini Speech Commands dataset containing audio recordings of spoken commands.
3. Organize the audio recordings into training and validation datasets and identify their class labels.
4. Convert the audio waveforms into spectrograms using the Short-Time Fourier Transform (STFT).
5. Visualize a sample spectrogram to examine the time-frequency representation of the audio.
6. Construct a CNN using convolutional, pooling, and dense layers for audio classification.
7. Compile and train the CNN using the training spectrograms.
8. Evaluate the CNN on the validation dataset and calculate accuracy, precision, recall, and F1-score.
9. Load the pretrained Whisper model to perform Speech-to-Text conversion on audio samples.
10. Convert the audio recordings into text and calculate the Word Error Rate (WER) by comparing the recognized text with reference labels.
11. Load the pretrained SpeechT5 text-to-speech model, vocoder, and a compatible speaker embedding.
12. Convert the input text into speech using the TTS model.
13. Save the generated speech as a WAV file and play the audio output.
14. Display the classification metrics, WER, and generated speech output.
15. Stop the program.

Code:

```
#1 Install libraries
!pip install -U datasets transformers soundfile
```

Show hidden output

```
#2 Import libraries and download the dataset
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt
import pathlib
import glob
import os
import soundfile as sf

from sklearn.metrics import (
    accuracy_score, precision_score,
    recall_score, f1_score,
    classification_report
)

# This downloads and extracts the content into cache_dir/cache_subdir
# which resolves to ./data/
_ = tf.keras.utils.get_file(
    "mini_speech_commands.zip",
    origin="http://storage.googleapis.com/download.tensorflow.org/data/mini_speech_commands.zip",
    extract=True,
    cache_dir=".",
    cache_subdir="data"
)

# Set directory to the extracted subdirectory containing the classes
data_dir = pathlib.Path("data/mini_speech_commands_extracted/mini_speech_commands")

print("Dataset path exists:", data_dir.exists())
print("Folders:", [p.name for p in data_dir.iterdir() if p.is_dir()])
```

```
#3 Load audio training and validation sets
BATCH_SIZE = 32
SEED = 42

train_audio = tf.keras.utils.audio_dataset_from_directory(
    data_dir,
    batch_size=BATCH_SIZE,
    validation_split=0.2,
    subset="training",
    seed=SEED,
    output_sequence_length=16000
)

test_audio = tf.keras.utils.audio_dataset_from_directory(
    data_dir,
    batch_size=BATCH_SIZE,
    validation_split=0.2,
    subset="validation",
    seed=SEED,
    output_sequence_length=16000
)

class_names = train_audio.class_names
num_classes = len(class_names)

print("Classes:", class_names)
print("Number of classes:", num_classes)
```

Show hidden output

```python
#4 Convert audio into spectrograms
def get_spectrogram(waveform, label):
    waveform = tf.squeeze(waveform, axis=-1)

    spectrogram = tf.abs(
        tf.signal.stft(
            waveform,
            frame_length=256,
            frame_step=128
        )
    )
    spectrogram = spectrogram[..., tf.newaxis]
    return spectrogram, label

train_ds = train_audio.map(get_spectrogram, num_parallel_calls=tf.data.AUTOTUNE).prefetch(tf.data.AUTOTUNE)

test_ds = test_audio.map(get_spectrogram,num_parallel_calls=tf.data.AUTOTUNE).prefetch(tf.data.AUTOTUNE)
```

```python
#5 Visualize one spectrogram
try:
    for audio_batch, label_batch in train_ds.take(1):
        plt.figure(figsize=(8, 4))
        plt.imshow(
            tf.math.log(audio_batch[0, :, :, 0] + 1e-6).numpy(),
            aspect="auto",
            origin="lower"
        )
        plt.title("Speech Spectrogram: " + class_names[int(label_batch[0])])
        plt.xlabel("Time Frame")
        plt.ylabel("Frequency Bin")
        plt.colorbar()
        plt.show()
except Exception as e:
    print("Error reading dataset. Please update data_dir in Cell #2 to:")
    print('data_dir = pathlib.Path("data/mini_speech_commands_extracted/mini_speech_commands")')
    print("and re-execute Cells #2, #3, and #4 before running this cell again.\n")
    raise e
```

```python
#6 Build the CNN audio classifier
audio_model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(124, 129, 1)),

    tf.keras.layers.Conv2D(16, 3, activation="relu"),
    tf.keras.layers.MaxPooling2D(),

    tf.keras.layers.Conv2D(32, 3, activation="relu"),
    tf.keras.layers.MaxPooling2D(),

    tf.keras.layers.Conv2D(64, 3, activation="relu"),
    tf.keras.layers.GlobalAveragePooling2D(),

    tf.keras.layers.Dense(64, activation="relu"),
    tf.keras.layers.Dropout(0.3),
    tf.keras.layers.Dense(num_classes, activation="softmax")
])

audio_model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

audio_model.summary()
```

Show hidden output

```python
#7 Train the audio classifier
history = audio_model.fit(
    train_ds,
    validation_data=test_ds,
    epochs=5
)
```

```python
#8 Evaluate classification performance
y_true = []
y_pred = []

for spectrograms, labels in test_ds:
    probabilities = audio_model.predict(spectrograms, verbose=0)
    predictions = np.argmax(probabilities, axis=1)

    y_true.extend(labels.numpy())
    y_pred.extend(predictions)

audio_accuracy = accuracy_score(y_true, y_pred)
audio_precision = precision_score(y_true, y_pred, average="macro", zero_division=0)
audio_recall = recall_score(y_true, y_pred, average="macro", zero_division=0)
audio_f1 = f1_score(y_true, y_pred, average="macro", zero_division=0)

print("Accuracy:", round(audio_accuracy, 4))
print("Macro Precision:", round(audio_precision, 4))
print("Macro Recall:", round(audio_recall, 4))
print("Macro F1-score:", round(audio_f1, 4))

print(classification_report(
    y_true, y_pred,
    labels=list(range(num_classes)),
    target_names=class_names,
    zero_division=0
))
```

Show hidden output

```python
#9 Plot training and validation accuracy
plt.figure(figsize=(8, 4))

plt.plot(history.history["accuracy"], marker="o", label="Training Accuracy")
plt.plot(history.history["val_accuracy"], marker="o", label="Validation Accuracy")

plt.title("CNN Audio Classification Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()
plt.grid(True)
plt.show()
```

```python
#10 Speech-to-Text (STT) using pretrained Whisper
import torch
import re
from transformers import pipeline
from jiwer import wer

wav_files = glob.glob(str(data_dir / "*" / "*.wav"))

print("Available audio files:", len(wav_files))

asr = pipeline(
    "automatic-speech-recognition",
    model="openai/whisper-tiny.en",
    device=0 if torch.cuda.is_available() else -1
)

def normalize_text(text):
    return re.sub(r"[^a-z0-9 ]", "", text.lower()).strip()

references = []
transcriptions = []

# Test on up to 10 audio clips
for path in wav_files[:10]:
    reference = pathlib.Path(path).parent.name
    result = asr(path)
    transcription = result["text"].strip()

    references.append(normalize_text(reference))
    transcriptions.append(normalize_text(transcription))

    print("Actual:", reference, "| Recognized:", transcription)

stt_wer = wer(references, transcriptions)

print("\nWord Error Rate (WER):", round(stt_wer, 4))
```

```python
# 11. Load pretrained Text-to-Speech models

import torch
import soundfile as sf
import numpy as np
from IPython.display import Audio, display
from transformers import (
    SpeechT5Processor,
    SpeechT5ForTextToSpeech,
    SpeechT5HifiGan
)
from datasets import load_dataset

# Select device
tts_device = "cuda" if torch.cuda.is_available() else "cpu"
print("Using device:", tts_device)

# Load pretrained models
tts_processor = SpeechT5Processor.from_pretrained("microsoft/speecht5_tts")

tts_model = SpeechT5ForTextToSpeech.from_pretrained("microsoft/speecht5_tts").to(tts_device)

vocoder = SpeechT5HifiGan.from_pretrained("microsoft/speecht5_hifigan").to(tts_device)

# Load the Parquet-compatible speaker dataset
speaker_dataset = load_dataset("Matthijs/cmu-arctic-xvectors",revision="refs/convert/parquet",split="validation")

# Select a genuine pretrained speaker embedding
speaker_embeddings = torch.tensor(
    speaker_dataset[0]["xvector"],
    dtype=torch.float32
).unsqueeze(0).to(tts_device)

print("Speaker embedding shape:", speaker_embeddings.shape)
print("TTS models and speaker embedding loaded successfully!")
```

```python
# 12. Generate and play speech

# Use Whisper's recognized text if available
text_to_speak = next(
    (text for text in transcriptions if text.strip()),
    "Hello, welcome to the audio classification application."
)

# Prepare text input
tts_inputs = tts_processor(text=text_to_speak, return_tensors="pt")

input_ids = tts_inputs["input_ids"].to(tts_device)

# Generate speech
with torch.no_grad():
    speech = tts_model.generate_speech(input_ids, speaker_embeddings, vocoder=vocoder)

# Convert to NumPy
speech_audio = speech.cpu().numpy()

# Save audio
output_file = "generated_speech.wav"

sf.write(output_file,speech_audio,samplerate=16000)

print("Text spoken:", text_to_speak)
print("Generated audio saved as:", output_file)

# Play generated speech
display(Audio(speech_audio, rate=16000))
```

```python
#13 Final evaluation summary
print("AUDIO CLASSIFICATION RESULTS")
print("----------------------------")
print("Accuracy :", round(audio_accuracy, 4))
print("Precision:", round(audio_precision, 4))
print("Recall   :", round(audio_recall, 4))
print("F1-score :", round(audio_f1, 4))

print("\nSPEECH-TO-TEXT RESULTS")
print("----------------------")
print("Word Error Rate:", round(stt_wer, 4))

print("\nTEXT-TO-SPEECH")
print("--------------")
print("Generated audio file: generated_speech.wav")
```

Output:

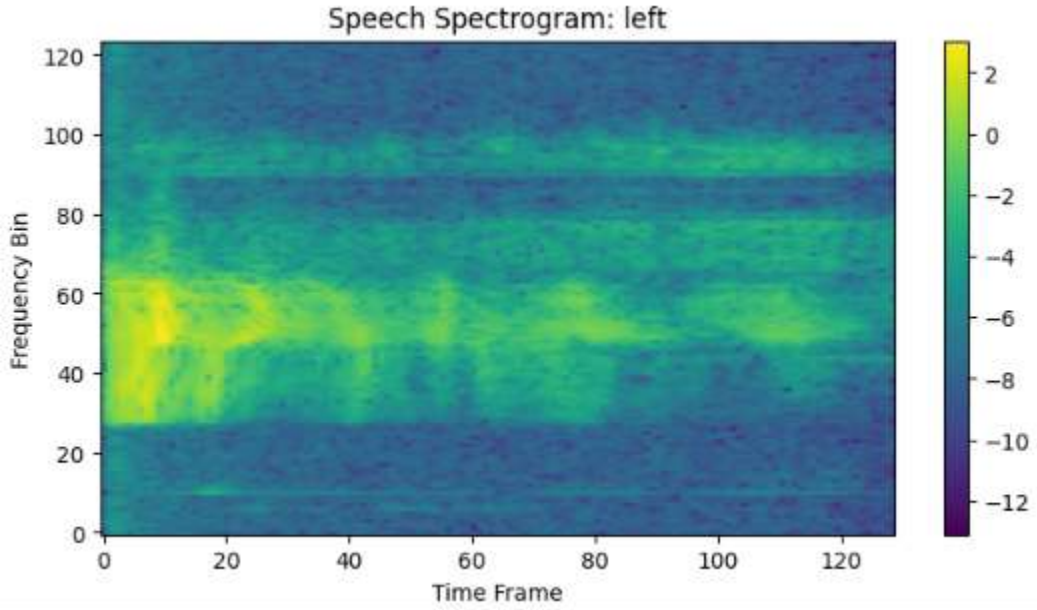
#2 Import libraries and download the dataset

```
Dataset path exists: True
Folders: ['right', 'go', 'no', 'stop', 'down', 'up', 'yes', 'left']
```

#3 Load audio training and validation sets

```
Found 8000 files belonging to 8 classes.
Using 6400 files for training.
Found 8000 files belonging to 8 classes.
Using 1600 files for validation.
Classes: ['down', 'go', 'left', 'no', 'right', 'stop', 'up', 'yes']
Number of classes: 8
```

#5 Visualize one spectrogram



#6 Build the CNN audio classifier

Model: "sequential_1"

Layer (type)	Output Shape	Param #
conv2d_3 (Conv2D)	(None, 122, 127, 16)	160
max_pooling2d_2 (MaxPooling2D)	(None, 61, 63, 16)	0
conv2d_4 (Conv2D)	(None, 59, 61, 32)	4,640
max_pooling2d_3 (MaxPooling2D)	(None, 29, 30, 32)	0
conv2d_5 (Conv2D)	(None, 27, 28, 64)	18,496
global_average_pooling2d_1 (GlobalAveragePooling2D)	(None, 64)	0
dense_2 (Dense)	(None, 64)	4,160
dropout_1 (Dropout)	(None, 64)	0
dense_3 (Dense)	(None, 8)	520

Total params: 27,976 (109.28 KB)
Trainable params: 27,976 (109.28 KB)
Non-trainable params: 0 (0.00 B)

#7 Train the audio classifier

```
Epoch 1/5
200/200 ━━━━━━━━━━━━━━━━━━━━ 158s 739ms/step - accuracy: 0.1758 - loss: 2.0321 - val_accuracy: 0.2488 - val_loss: 1.9169
Epoch 2/5
200/200 ━━━━━━━━━━━━━━━━━━━━ 89s 446ms/step - accuracy: 0.2577 - loss: 1.8671 - val_accuracy: 0.2869 - val_loss: 1.8070
Epoch 3/5
200/200 ━━━━━━━━━━━━━━━━━━━━ 144s 455ms/step - accuracy: 0.3378 - loss: 1.7440 - val_accuracy: 0.4094 - val_loss: 1.6417
Epoch 4/5
200/200 ━━━━━━━━━━━━━━━━━━━━ 87s 434ms/step - accuracy: 0.3963 - loss: 1.6197 - val_accuracy: 0.4325 - val_loss: 1.5508
Epoch 5/5
200/200 ━━━━━━━━━━━━━━━━━━━━ 141s 432ms/step - accuracy: 0.4433 - loss: 1.5233 - val_accuracy: 0.4706 - val_loss: 1.4966
```

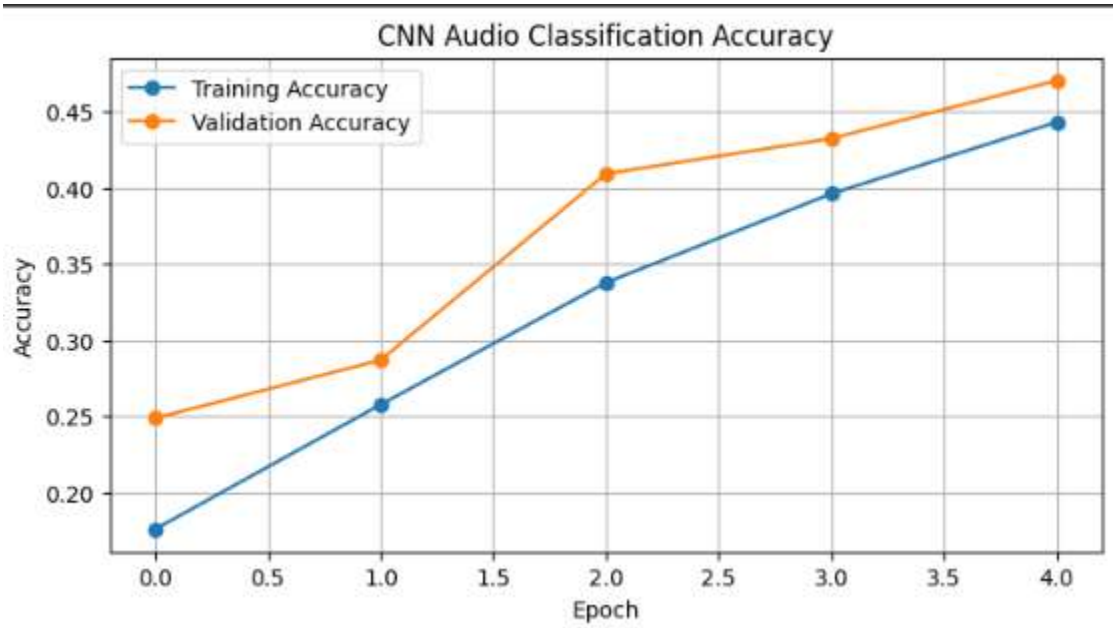
#8 Evaluate classification performance

```
Accuracy: 0.4706
Macro Precision: 0.5037
Macro Recall: 0.4803
Macro F1-score: 0.4611
              precision    recall  f1-score   support

        down       0.56      0.66      0.61       185
          go       0.44      0.61      0.51       188
        left       0.42      0.30      0.35       207
          no       0.44      0.38      0.41       207
       right       0.63      0.23      0.34       204
        stop       0.48      0.39      0.43       218
          up       0.37      0.84      0.52       183
         yes       0.69      0.43      0.53       208

    accuracy                           0.47      1600
   macro avg       0.50      0.48      0.46      1600
weighted avg       0.51      0.47      0.46      1600
```

#9 Plot training and validation accuracy



#13 Final evaluation summary

```
AUDIO CLASSIFICATION RESULTS
----------------------------
Accuracy : 0.4706
Precision: 0.5037
Recall   : 0.4803
F1-score : 0.4611

SPEECH-TO-TEXT RESULTS
----------------------
Word Error Rate: 0.1

TEXT-TO-SPEECH
--------------
Generated audio file: generated_speech.wav
```